

الهدف العام

بناء منصة Autonomous AI Software Engineering متقدمة قادرة على تحويل أي فكرة أو طلب يكتبه المستخدم باللغة الطبيعية إلى مشروع برمجي حقيقي Production-ready، مهما كان مستوى تعقيده، بدءًا من فهم الهدف والمتطلبات، مرورًا بتحليل المنتج، تصميم UX/UI، اختيار Architecture، تصميم قاعدة البيانات، التخطيط، كتابة الكود، تشغيل المشروع، استخدام Browser و Terminal و Filesystem، ربط الخدمات الخارجية، اختبار التطبيق، اكتشاف الأخطاء، تحليل أسبابها، إصلاحها، إعادة الاختبار، فحص الأمن والأداء و Accessibility و SEO، نشر المشروع، مراقبته، وصيانتته وتطويره باستمرار.

يجب ألا تكون المنصة Chatbot أو AI Code Generator أو Website Builder تقليديًا.

يجب أن تكون Autonomous AI Software Engineering Platform تمتلك AI Brain، Agent Runtime، Computer Use، Browser، Terminal، Sandbox، Memory، Context Engineering، Knowledge Base، Tool System، Continuous، Orchestration، Testing، Self-Healing، Security، Deployment، Monitoring، Evaluation، وImprovement.

1. Design System

إنشاء Design System احترافي وقابل للتوسع يحتوي على:

Design Tokens، Colors، Typography، Font System، Spacing، Grid، Borders، Radius، Shadows، Elevation، Icons، Motion، Animation، Responsive Breakpoints، Dark Mode، Light Mode، Accessibility، Visual States، Component Guidelines، Layout Rules، Themes، Visual Hierarchy.

يجب ألا يكون النظام مقيدًا بتصميم واحد.

يجب أن يستطيع إنشاء مشاريع مختلفة جذريًا من حيث الشكل والأسلوب والهوية البصرية.

يجب استخدام نظام Tokens موحد بحيث يمكن تغيير التصميم على مستوى المشروع بالكامل.

يمكن استخدام Tailwind CSS و shadcn/ui و Radix UI و Lucide أو بدائل مفتوحة المصدر.

2. UX System

يجب توفير UX Architecture كاملة تشمل:

User Flows, Information Architecture, Navigation, Search, Filtering, Sorting, Forms, Validation, Onboarding, Empty States, Loading States, Error States, Success States, Confirmation Flows, Notifications, Modals, Drawers, Keyboard Navigation, Mobile UX, Responsive UX, Progressive Disclosure, Undo/Redo, Contextual Actions.

يجب تحليل رحلة المستخدم قبل إنشاء الواجهة.

3. UX Quality Engine

يجب أن يستطيع النظام تقييم:

وضوح User Flow, Visual Hierarchy, Navigation, عدد الخطوات, سهولة استخدام Forms, جودة Empty States, Loading States, Error States, Mobile Experience, Consistency, Accessibility, Conversion Flow.

ثم يقترح أو ينفذ التحسينات.

4. Component System

إنشاء مكتبة Components متكاملة تشمل:

Button, Input, Select, Dropdown, Modal, Drawer, Tabs, Card, Table, Data Grid, Form, Calendar, Command Menu, Tooltip, Popover, Toast, Alert, Badge, Avatar, Sidebar, Navbar, Breadcrumb, Pagination, Chart, File Upload, Rich Text Editor, Code Editor, Terminal, Logs, Chat, AI Panel, Kanban, Timeline, Wizard, Data Table, Filters وغيرها.

كل Component يجب أن يدعم:

Default, Hover, Focus, Active, Disabled, Loading, Error, Success, Selected, Empty, Read-only, Responsive, Dark Mode.

5. Frontend Engineering

دعم React و TypeScript و Vite أو Next.js حسب طبيعة المشروع.

يجب دعم:

Component Architecture, Routing, State Management, Server State, Client State, Forms, Validation, Data Fetching, Caching, Optimistic Updates, Error Boundaries, Loading UI, Responsive Design, Accessibility, SEO, Performance, Code Splitting, Lazy Loading, Image Optimization, Animations, Browser Compatibility, Progressive Enhancement.

6. Backend Engineering

إنشاء Backend Production-ready يشمل:

Authentication, Authorization, Sessions, APIs, Validation, Rate Limiting, File Upload, Storage, Email, Notifications, Payments, Webhooks, Background Jobs, Queues, Cron Jobs, Audit Logs, Error Handling, Logging, Monitoring, External Integrations.

يجب فصل Business Logic عن Routes و Controllers و Database.

7. Database Engineering

دعم PostgreSQL وقواعد البيانات المناسبة.

يشمل:

Schema Design, Relationships, Keys, Constraints, Indexes, Transactions, Migrations, Seeds, RLS, Query Optimization, Pagination, Search, Full-text Search, Backups, Data Integrity, Soft Delete, Audit Data, Database Security.

يجب تصميم Schema وعلاقات البيانات قبل بناء Business Logic.

8. API System

دعم:

REST, tRPC, WebSockets, Webhooks, GraphQL عند الحاجة.

يشمل:

Validation, Authentication, Authorization, Error Standards, Rate Limiting, Versioning, Documentation, Retries, Timeouts, Idempotency, Webhook Verification, Logging, Monitoring.

9. Authentication & Identity

دعم:

Email/Password, OAuth, Google Login, GitHub Login, MFA, Sessions, Password Reset, Email Verification, Roles, Permissions, Organizations, Teams, Invitations, API Keys, Service Accounts, Device Management, Session Revocation.

10. Authorization & RBAC

بناء Permission Engine:

Users → Organizations → Teams → Roles → Permissions → Resources → Actions.

يجب أن تكون الصلاحيات قابلة للتخصيص لكل مشروع.

11. Requirements Engineering

تحويل طلب المستخدم إلى:

Intent → Requirements → User Stories → Acceptance Criteria → Functional Requirements → Non-functional Requirements → Constraints → Dependencies → Risks → Technical Specification.

يجب اكتشاف الغموض والتعارضات قبل بدء التنفيذ.

12. Requirement Conflict Detection

يجب اكتشاف التعارضات مثل:

طلب Feature يتعارض مع Security، أو Requirement يتعارض مع Architecture، أو Feature تتطلب API غير متاح.

يجب تنبيه المستخدم أو إعادة التخطيط.

13. Product Intelligence

يجب تحليل المنتج واقتراح:

Missing Features, UX Improvements, Security Improvements, Performance Improvements, SEO Improvements, Accessibility Improvements, Testing Gaps.

لكن لا يجب تغيير Product Requirements الجوهرية دون موافقة المستخدم.

14. Architecture Engine

قبل البرمجة يجب تحديد:

Frontend Architecture, Backend Architecture, Database Architecture, API Architecture, Authentication, Authorization, State Management, Storage, Infrastructure, Integrations, Testing, Security, Deployment.

يجب اختيار Architecture بناءً على المشروع وليس استخدام Architecture واحدة دائمًا.

15. Architecture Decision Records

يجب حفظ القرارات المعمارية:

ما القرار؟ لماذا تم اختياره؟ ما البدائل؟ ما العيوب؟ وما تأثير القرار؟
ويجب أن يستطيع الـ Agent الرجوع إليها لاحقًا.

16. Planning Engine

يجب إنشاء خطة تنفيذ حقيقية:

Milestones → Tasks → Dependencies → Parallel Tasks → Critical Path → Quality Gates.

يجب أن تكون كل مهمة قابلة للتنفيذ والتحقق.

17. Task Graph

تحويل المشروع إلى Dependency Graph بحيث يستطيع النظام معرفة:

ما الذي يجب تنفيذه أولاً؟

ما الذي يمكن تنفيذه بالتوازي؟

ما الذي يعتمد على ماذا؟

ما الذي يجب إعادة تنفيذه بعد تغيير معين؟

18. Agent Orchestrator

وجود Orchestrator مركزي يدير جميع Agents والمهام.

الحالات:

IDLE → ANALYZING → PLANNING → WAITING_FOR_INPUT → EXECUTING → TESTING → FIXING → VERIFYING → AWAITING_APPROVAL → DEPLOYING → MONITORING → COMPLETED / FAILED / PAUSED / CANCELLED.

19. Agent Runtime Kernel

نواة تشغيل الوكلاء مسؤولة عن:

Agent Lifecycle, Task Lifecycle, State, Context, Tool Access, Memory, Checkpoints, Timeouts, Retries, Permissions, Handoffs, Cancellation.

20. Specialized Agents

يجب توفير:

Requirements Agent
Product Agent
Planner Agent
Architect Agent
UI/UX Agent
Frontend Agent
Backend Agent
Database Agent
Integration Agent
Testing Agent
Browser QA Agent
Security Agent
Performance Agent
SEO Agent
Accessibility Agent

Code Review Agent
Debug Agent
DevOps Agent
Deployment Agent
Monitoring Agent
Maintenance Agent
Research Agent
Documentation Agent
Refactoring Agent
Migration Agent

21. Agent Communication

يجب أن يكون التواصل منظمًا عبر:

Task, Context, Inputs, Outputs, Artifacts, Dependencies, Errors, Status, Confidence, Handoff.

يجب منع التواصل العشوائي بين الوكلاء.

22. Parallel Agent Execution

المهام المستقلة يجب أن تعمل بالتوازي.

يجب وجود Conflict Detection لمنع Agents من تعديل نفس الملفات أو الموارد بطريقة متعارضة.

23. Dynamic Replanning

الدورة ليست:

Plan → Execute فقط.

بل:

Plan → Execute → Observe → Evaluate → Replan → Continue.

إذا ظهرت مشكلة يجب إعادة التخطيط للجزء المتأثر.

24. Context Engineering

إدارة Context بذكاء من خلال:

Context Selection, File Retrieval, Code Retrieval, Dependency Context, Project Context, Conversation Context, Summarization, Compression, Token Budgeting, Relevance Ranking.

لا يتم إرسال كامل المشروع إلى النموذج في كل خطوة.

25. Codebase Intelligence

فهرسة المشروع وفهم:

Files, Symbols, Imports, Exports, Components, Routes, APIs, Database Relations, Tests, Services, Configurations, Dependencies.

يجب معرفة تأثير كل تغيير على بقية النظام.

26. Semantic Code Search

البحث داخل المشروع بالمعنى وليس فقط Exact Text Search.

مثلاً:

"أين تتم معالجة عملية الدفع؟"

يجب أن يجد النظام الملفات والمكونات والـ API والـ Database المرتبطة.

27. Dependency Graph

إنشاء Graph يربط:

Components → Files → APIs → Services → Database → Tests.

يستخدم لتحديد تأثير التغييرات.

28. Memory System

الفصل بين:

Short-Term Memory

Project Memory

User Preferences

Task Memory

Technical Decisions

Error Memory

Tool Memory.

يجب أن تستمر الذاكرة بين المهام.

29. Knowledge Base

توفير:

Framework Documentation, API Documentation, Coding Patterns, Architecture Patterns, UI Patterns, Security Rules, Testing Patterns, Database Patterns, Deployment Patterns, Error Knowledge, Solutions, Code Examples, Templates, Best Practices.

يجب استخدام Retrieval للحصول على المعلومات المناسبة.

30. Web Research Agent

يجب أن يستطيع النظام البحث في الإنترنت عند الحاجة إلى معلومات حديثة أو Documentation أو APIs أو Libraries أو Services.

يجب جمع المصادر والتحقق منها قبل اعتماد المعلومات.

31. Source Verification

يجب تقييم المصادر والتأكد من أن المعلومات المستخدمة حديثة وموثوقة ومناسبة للمهمة.

32. Tool System

يجب أن يمتلك الـ Agent أدوات حقيقية:

Browser, Terminal, Filesystem, Git, Database, API Client, Search, Documentation, Screenshot, Test Runner, Build System, Docker, SSH, Deployment, Monitoring, Storage.

33. Tool Discovery

يجب أن يستطيع النظام اكتشاف الأدوات المناسبة للمهمة بدل إعطاء كل Agent كل الأدوات.

34. Tool Permission Engine

كل Tool يجب أن يدعم:

ALLOW / DENY / ASK USER.

يجب تطبيق Least Privilege.

35. Computer-Use System

يجب أن يستطيع Agent التعامل مع الكمبيوتر والواجهات الرسومية:

Mouse, Keyboard, Browser, Windows, Applications, Files, Screenshots, Drag & Drop, Clipboard, Downloads, Uploads.

36. Browser Agent

متصفح حي يدعم:

Tabs, Navigation, Login, Cookies, Sessions, Forms, JavaScript, Uploads, Downloads, Screenshots, Console, Network, Responsive Viewports, Multi-step Workflows.

37. Browser Profiles

دعم Sessions و Profiles منفصلة مع Cookies و Storage و Authentication State، مع عزل المشاريع والمستخدمين.

38. Visual DOM Understanding

يجب الجمع بين:

DOM + Accessibility Tree + Screenshot + Visual Understanding.

حتى يستطيع الـ Agent فهم ما يراه وما يمكن التفاعل معه.

39. Terminal Agent

يجب أن يستطيع:

Install Packages, Run Commands, Build, Test, Start Server, Inspect Logs, Run Migrations, Run Git, Lint, Format, Security Scan.

40. Filesystem Agent

دعم:

Create, Read, Edit, Search, Rename, Move, Delete, Diff, Compare, Refactor, Organize.

مع تتبع جميع التغييرات.

41. Sandbox Execution

كل مشروع أو مهمة خطيرة يجب أن تعمل داخل Sandbox مع:

Filesystem Isolation, Process Isolation, Network Policies, CPU Limits, RAM Limits, Disk Limits, Timeouts, Containers, Cleanup, Logs.

42. Execution Plane

يجب فصل:

Control Plane

عن:

Execution Plane.

يجب ألا يستطيع كود المشروع الوصول مباشرة إلى أسرار النظام أو Control Plane.

43. Network Egress Control

تحديد الخدمات والمواقع التي يمكن للSandbox الاتصال بها.

44. Filesystem Policies

تحديد:

أي ملفات يمكن قراءتها؟

أي ملفات يمكن تعديلها؟

أي ملفات ممنوع الوصول إليها؟

45. Secrets Vault

حماية:

API Keys, OAuth Tokens, SSH Keys, Database Credentials, Environment Variables.

يجب ألا يتم كشف Secrets للـ AI إلا عند الضرورة وبصلاحيات محددة.

46. OAuth Platform

دعم OAuth وScopes وإدارة Connections للخدمات الخارجية.

47. Integrations Platform

دعم تكاملات مثل:

GitHub, Google, Gmail, Slack, Discord, Notion, Stripe, Supabase, Cloud Providers, Storage, CRM, Calendar, WhatsApp وغيرها.

48. Artifact System

كل مهمة تنتج Artifacts قابلة للتتبع:

Code, Files, Screenshots, Reports, Tests, Logs, Database Schemas, Builds, Deployments, Documents.

49. Checkpoint System

أخذ Checkpoints قبل وبعد العمليات المهمة.

50. Resume System

يجب أن يستطيع النظام:

Pause → Resume

مع الاحتفاظ بالحالة والـ Context والـ Artifacts والـ Tasks.

51. Failure Recovery

التعامل مع:

API Failure, Model Failure, Network Failure, Timeout, Browser Crash, Container Crash, Server Failure, Database Failure, Deployment Failure, Rate Limits.

مع:

Retry, Backoff, Fallback, Recovery, Rollback.

52. Self-Healing

الدورة:

Detect → Diagnose → Fix → Retest → Verify.

يجب البحث عن Root Cause وليس فقط معالجة أعراض المشكلة.

53. Regression Testing

بعد كل إصلاح يجب التأكد من عدم كسر وظائف موجودة.

54. Code Review Engine

تحليل:

Architecture, Readability, Maintainability, Security, Performance, Type Safety, Duplication, Complexity, Error Handling, Testing Coverage.

55. Refactoring Engine

يجب تحسين:

Duplicate Code, Dead Code, Complexity, Naming, Structure, Performance.

بدون تغيير Behavior.

56. Technical Debt Detection

يجب اكتشاف Technical Debt وتصنيفها حسب:

Severity, Impact, Risk, Effort.

57. Migration Engine

دعم:

Database Migrations, Dependency Upgrades, Framework Upgrades, API Migrations.

مع Backup و Testing و Rollback.

58. API Contract Testing

التأكد من توافق:

Frontend ↔ Backend ↔ External APIs.

59. Schema Compatibility

يجب اكتشاف التغييرات الخطرة على Database Schema قبل تنفيذها.

60. Testing System

دعم:

Unit, Component, Integration, API, E2E, Browser, Regression, Accessibility, Visual, Performance, Security, Database, Smoke, Contract Tests.

61. Synthetic User Testing

إنشاء User Personas افتراضية لتنفيذ User Flows واكتشاف مشاكل UX.

62. Adversarial Testing

محاولة كسر النظام واكتشاف:

Security Bugs, Logic Bugs, Permission Bugs, Input Vulnerabilities, Workflow Failures.

63. Prompt Injection Defense

يجب حماية الـ Agent من التعليمات الخبيثة الموجودة داخل:

Web Pages, Emails, Documents, Code, Files, APIs.

المحتوى الخارجي يجب اعتباره Untrusted.

64. Tool Output Sanitization

يجب التحقق من نتائج الأدوات قبل إدخالها إلى Agent Context.

65. Security Engine

يجب حماية المشروع من:

XSS, CSRF, SQL Injection, SSRF, Command Injection, Path Traversal, Authentication Attacks, Authorization Bugs, Secret Leakage, Dependency Vulnerabilities, Session Attacks, Rate Abuse.

66. Dependency Security

فحص:

Vulnerabilities, Licenses, Supply Chain Risks, Malicious Packages, Outdated Dependencies.

67. SBOM

توليد Software Bill of Materials للمشاريع Production.

68. Privacy Engine

اكتشاف وحماية البيانات الحساسة والشخصية.

69. PII Detection

اكتشاف:

Names, Emails, Phone Numbers, Addresses, IDs داخل Logs وFiles وغيرها من البيانات الشخصية داخل Databases.

70. Data Retention

تحديد مدة الاحتفاظ بالبيانات وآليات حذفها بأمان.

71. Performance Engine

تحسين:

JavaScript, Images, Fonts, Network, API, Database, Server, Caching, CDN.

قياس:

Core Web Vitals, Bundle Size, API Latency, Database Performance, Server Response Time.

72. Automatic Performance Optimization

اكتشاف Bottlenecks واقتراح أو تنفيذ التحسينات بعد التحقق.

73. SEO Engine

دعم:

Metadata, Titles, Descriptions, Canonical URLs, Sitemap, Robots, Structured Data, Open Graph, Semantic HTML, Internal Linking, Image Alt, SSR/SSG.

74. Accessibility Engine

دعم WCAG المناسب للمشروع.

يشمل:

Keyboard, Screen Reader, Focus, Contrast, Semantic HTML, ARIA, Forms, Error Messages, Reduced Motion.

75. Accessibility Auto-Fix

اكتشاف المشاكل وإصلاحها ثم إعادة الاختبار.

76. Visual QA Engine

تحليل Screenshots لاكتشاف:

Alignment, Overflow, Spacing, Typography, Responsive Problems, Broken Components, Visual Inconsistency.

ثم:

Analyze → Fix → Screenshot → Compare.

77. Responsive Intelligence

اختبار:

Mobile, Tablet, Laptop, Desktop, Large Screens.

واكتشاف أفضل Layout لكل Viewport.

78. Design-to-Code

تحويل التصميم إلى:

Components, Tokens, Layout, Responsive Rules, Assets.

79. Code-to-Design Analysis

تحليل المشروع لمعرفة مدى التزامه بالـDesign System.

80. Localization Engine

دعم:

Languages, RTL, LTR, Currency, Date Formats, Timezones, Number Formats.

81. Arabic / RTL Engine

يجب أن يكون دعم العربية حقيقيًا، وليس مجرد direction: rtl.

يجب دعم:

Typography, Layout, Icons, Tables, Forms, Navigation, Mixed RTL/LTR Content.

82. DevOps Engine

Pipeline:

Git → Branch → Lint → Type Check → Tests → Security Scan → Build → Docker → Deploy → Health Check → Smoke Test → Monitoring.

83. Infrastructure as Code

إدارة Infrastructure من خلال كود قابل للتكرار.

84. Deployment Engine

دعم:

SSH, Docker, Nginx, SSL, Domains, Environment Variables, Database, Containers, Health Checks, Smoke Tests, Rollback.

85. Environments

دعم:

Development → Staging → Production.

مع عزل Secrets و Databases و Deployments.

86. Canary Deployment

إمكانية نشر الإصدار الجديد على نسبة محدودة قبل التعميم.

87. Blue/Green Deployment

إمكانية تشغيل نسختين والتبديل بينهما بأمان.

88. Feature Flags

تشغيل وإيقاف Features بدون إعادة نشر.

89. Automatic Rollback

إذا فشل الإصدار الجديد أو ظهرت مشاكل، يتم الرجوع إلى الإصدار السابق.

90. Production Verification

بعد Deployment يجب تنفيذ:

Health Check → Database Check → Authentication → Core User Flows → APIs → Browser → Responsive → Security → Performance → Logs → Smoke Test.

91. Monitoring

مراقبة:

Application, Server, Database, APIs, Errors, Uptime, Resource Usage, Deployment, Performance.

92. Agent Observability

مراقبة:

Tokens, Cost, Latency, Tool Calls, Retries, Failures, Task Duration, Success Rate.

93. Incident Management

عند وقوع مشكلة Production:

Detect → Classify → Diagnose → Mitigate → Fix → Verify → Document.

94. Root Cause Analysis

يجب أن يبحث النظام عن السبب الجذري وليس مجرد الأعراض.

95. Alert Intelligence

تقليل Alerts المكررة وربط الأحداث ببعضها.

96. SLO / SLI

قياس Availability و Reliability و Latency وفق مؤشرات واضحة.

97. Model Router

اختيار النموذج حسب:

Task Complexity, Cost, Latency, Capability, Context Size, Reliability.

98. Model Fallback

عند فشل النموذج الأساسي يتم التحويل إلى نموذج بديل.

99. Model Evaluation

اختبار النماذج قبل اعتمادها على المهام الحقيقية.

100. Agent Evaluation

اختبار Agents على مجموعة ثابتة من المهام وقياس:

Success Rate, Quality, Cost, Speed, Reliability.

101. A/B Agent Testing

مقارنة Agent Versions أو Strategies مختلفة على نفس المهام.

102. Regression Benchmark

بعد كل تحديث للنظام يجب تشغيل Benchmark للتأكد من عدم انخفاض الجودة.

103. Continuous Improvement

الدورة:

Task → Result → Evaluation → Feedback → Knowledge Update → Improvement.

104. Prompt Versioning

105. Agent Versioning

تشغيل إصدارات مختلفة من الـ Agents وإمكانية Rollback.

106. Cost Management

مراقبة:

Tokens, API Calls, Compute, Storage, Browser Time, Agent Runtime.

107. Budget Guardrails

يجب أن يستطيع المستخدم تحديد:

Maximum Cost, Maximum Time, Maximum Tool Calls, Maximum Compute.

إذا تم الوصول إلى الحد يتم إيقاف المهمة أو طلب الموافقة.

108. Prompt Caching

تقليل تكلفة Context المتكرر.

109. Semantic Cache

إعادة استخدام نتائج المهام المتشابهة عندما يكون ذلك آمنًا.

110. Distributed Task Scheduler

توزيع المهام على Workers.

111. Queue System

استخدام:

Task Queue → Worker → Execution → Result.

112. Event Bus

جميع الأنظمة يمكن أن تتواصل عبر Events مثل:

TaskCreated, AgentStarted, ToolCalled, FileChanged, TestFailed, DeploymentStarted, DeploymentCompleted.

113. Resource Manager

إدارة:

CPU, RAM, Disk, Network, Browser Sessions, Containers, Workers.

114. Scalability

تصميم المنصة لتدعم التوسع:

Workers, Queues, Caching, Horizontal Scaling, Database Scaling, CDN, Object Storage, Rate

Limiting.

115. Multi-Project System

كل مشروع يجب أن يمتلك:

Codebase, Memory, Context, Secrets, Database, Integrations, Environments, Deployments.

مع عزل كامل بين المشاريع.

116. Multi-Tenant Architecture

عند تحويل المنصة إلى SaaS:

Organizations, Users, Teams, Projects, Roles, Permissions, Billing, Usage, Quotas, Isolation.

117. Usage & Billing

دعم:

Subscriptions, Credits, Tokens, Compute, Storage, Agent Minutes, Model Costs, Quotas, Invoices.

118. Audit System

تسجيل:

User, Agent, Tool, Action, Time, Files, APIs, Result, Risk.

119. Forensics

عند حدوث مشكلة يجب القدرة على إعادة بناء:

ماذا حدث؟ لماذا حدث؟ أي Agent قام به؟ أي Tool؟ ما الملفات؟ وما النتيجة؟

120. Backup System

دعم:

Database Backup, Project Backup, Configuration Backup, Deployment Backup.

121. Disaster Recovery

يجب أن تكون هناك خطة آلية لاستعادة:

Database, Projects, Infrastructure, Configuration.

122. Reproducible Builds

يجب أن يمكن إعادة إنشاء نفس Build من نفس Source و Dependencies.

123. Environment Parity

تقليل الاختلاف بين:

Development, Staging, Production.

124. Documentation Engine

إنشاء وتحديث:

README, API Documentation, Architecture Documentation, Setup Guide, Deployment Guide, Changelog.

125. Release Management

إدارة:

Versions, Releases, Tags, Changelogs, Deployment History.

126. License Compliance

فحص تراخيص Dependencies والتأكد من توافقها مع المنتج.

127. Artifact Versioning

إصدار:

Screenshots, Reports, Builds, Documentation, Generated Assets.

128. Project Snapshotting

أخذ Snapshot كامل قبل التغييرات الحساسة.

129. Human-in-the-Loop

عند العمليات الحساسة يجب عرض:

Action, Reason, Risk, Files, Cost, Permissions, Expected Result.

ثم:

Approve / Reject / Modify.

130. Natural Language Interface

يجب أن يستطيع المستخدم قول:

"أضف نظام دفع."

"غيّر التصميم."

"أصلح المشكلة."

"انشر المشروع."

"أضف لوحة تحكم."

"اجعل الموقع RTL."

"راجع الأمان."

ويحول النظام الطلب إلى Tasks حقيقية.

131. Interactive Workspace

يجب أن تحتوي واجهة المنصة على:

Chat, Project Explorer, Agent Activity, Task Graph, Code Diff, Terminal, Browser Preview,

Screenshots, Logs, Tests, Deployments, Database, Integrations, Settings.

132. Real-Time Activity

عرض الأحداث الحقيقية:

Planning, Reading Files, Writing Code, Running Command, Running Tests, Browser Action, Error, Fixing, Deployment.

يمنع تمامًا إنشاء Activity وهمية.

133. Live Preview

يجب عرض التطبيق الحقيقي أثناء البناء مع:

Reload, Responsive Viewports, Console, Network, Screenshots, Browser Controls.

134. Project Collaboration

دعم:

Teams, Comments, Mentions, Reviews, Shared Projects, Permissions, Activity.

135. Product Analytics

بعد النشر يمكن دعم:

Funnels, Conversions, Retention, User Behavior, Feature Usage.

136. Feedback Loop

تحويل ملاحظات المستخدمين إلى:

Issue → Analysis → Task → Implementation → Test → Release.

137. Professional Templates

توفير Templates مرجعية لـ:

SaaS, E-commerce, Marketplace, CRM, ERP, Booking, Hotel, Restaurant, Learning, Social Network, Portfolio, Dashboard, AI Application, Admin Panel, API Platform, Internal Tool.

لكن Templates لا يجب أن تحد من قدرة النظام على بناء مشاريع جديدة.

138. AI Operating System Layer

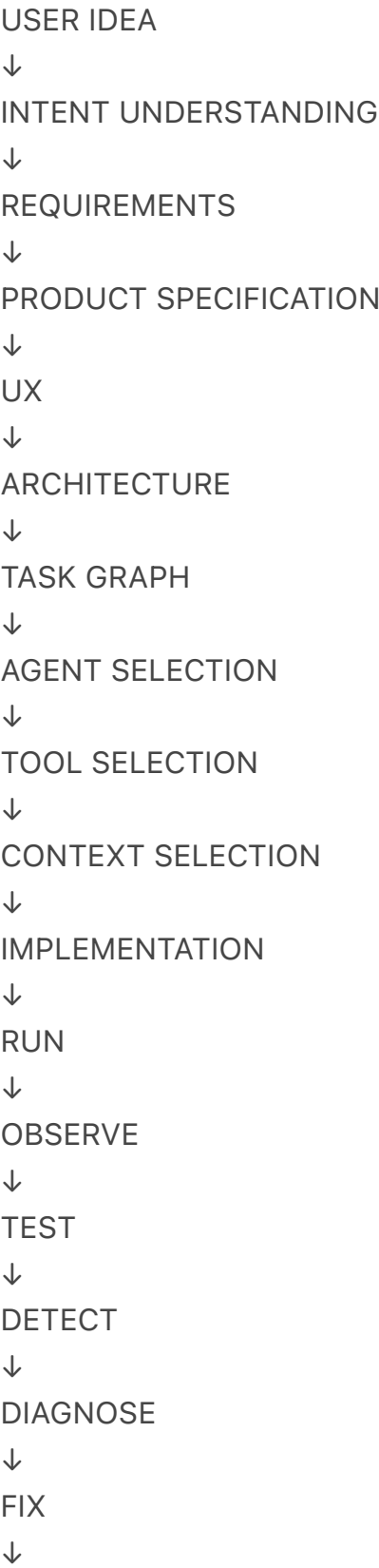
يجب أن تحتوي المنصة على Meta-Agent قادر على تحديد:

- أي Agent يجب استخدامه؟
- أي Model؟
- أي Tool؟
- أي Context؟
- أي Sandbox؟
- ما مستوى الصلاحية؟
- ما استراتيجية الاختبار؟
- متى يجب إعادة التخطيط؟
- متى يجب طلب موافقة المستخدم؟

أي أن النظام لا ينفذ فقط، بل يقرر كيف ينفذ.

139. Autonomous Development Loop

الدورة الرئيسية:



RETEST
↓
SECURITY AUDIT
↓
PERFORMANCE AUDIT
↓
ACCESSIBILITY AUDIT
↓
SEO AUDIT
↓
VISUAL QA
↓
CODE REVIEW
↓
QUALITY GATES
↓
VERIFY
↓
DEPLOY
↓
HEALTH CHECK
↓
MONITOR
↓
MAINTAIN
↓
LEARN
↓
IMPROVE

140. Core Architecture

يجب تقسيم المنصة إلى:

AI Control Plane

Models, Orchestrator, Planning, Memory, Context, Policies, Agent Management.

Agent Runtime

Tasks, Agents, State, Handoffs, Parallel Execution, Checkpoints, Recovery.

Tool Plane

Browser, Terminal, Filesystem, Git, Database, APIs, Search, Documentation, Deployment.

Execution Plane

Sandbox, Containers, Processes, Network, Files, Resources.

Engineering Plane

Frontend, Backend, Database, Testing, Security, Performance, DevOps.

Integration Plane

OAuth, APIs, GitHub, Cloud, Communication, Payments, Storage.

Observability Plane

Logs, Metrics, Traces, Agent Monitoring, Evaluation.

User Experience Plane

Chat, Dashboard, Workspace, Browser Preview, Code Editor, Terminal, Activity, Approval UI.

141. Core Safety Principles

يجب أن تكون جميع العمليات:

Secure, Isolated, Auditable, Reversible, Permissioned, Observable.

لا يجب أن يستطيع Agent تنفيذ عمليات حساسة خارج الصلاحيات المحددة.

لا يجب وضع Secrets في Context غير ضروري.

لا يجب تشغيل Code غير موثوق خارج Sandbox.

لا يجب نشر Production بدون Quality Gates.

لا يجب اعتبار المهمة ناجحة بناءً على رسالة النموذج فقط.

142. Core Quality Principle

المعيار الحقيقي للنجاح هو:

Evidence over Claims.

أي أن النظام لا يقول:

"تم بناء المشروع بنجاح"

إلا عندما يستطيع إثبات:

Build Passed
Tests Passed
Browser QA Passed
Visual QA Passed
Security Passed
Performance Acceptable
Deployment Healthy
Core User Flows Working

143. Final Objective

الهدف النهائي هو بناء منصة:

Autonomous AI Software Engineering Operating System

يمكن للمستخدم أن يعطيها فكرة بسيطة أو مشروعًا معقدًا، فتقوم بفهم الهدف، وتحليل المتطلبات، وتصميم المنتج، اختيار Architecture، إنشاء Database، بناء Frontend و Backend، استخدام الأدوات الحقيقية، البحث عند الحاجة، تشغيل Browser و Terminal، تنفيذ الاختبارات، اكتشاف الأخطاء، إصلاحها، فحص الأمن والأداء، نشر المشروع، مراقبته، وتطويره لاحقًا.

يجب أن تكون قادرة على بناء:

Websites, SaaS, E-commerce, Marketplaces, Dashboards, CRM, ERP, Booking Platforms, AI Applications, Internal Tools, APIs, Admin Systems, Mobile-ready Web Apps, ومشاريع مخصصة بالكامل.

ويجب ألا يكون هناك قالب بصري أو Architecture واحدة تفرض على جميع المشاريع.

يجب أن تكون المنصة:

Flexible + Autonomous + Secure + Observable + Testable + Reversible + Scalable + Production-ready.

والدورة النهائية التي يجب أن تحكم النظام بالكامل هي:

**UNDERSTAND → PLAN → ARCHITECT → BUILD → EXECUTE → OBSERVE → TEST → DETECT →
DIAGNOSE → FIX → RETEST → REVIEW → SECURE → OPTIMIZE → VERIFY → DEPLOY →
MONITOR → MAINTAIN → LEARN → IMPROVE**

هذه هي المواصفة المستهدفة لمنصة AI Software Engineering من الفئة المتقدمة، وليست مجرد Website Generator.